

FRONTEND IMPLEMENTATION GUIDE: Template-Based Dynamic Styling

Overview

The backend now supports template-based styling for dynamically generated reports. The frontend needs to be updated to allow users to upload a style template, activate it, and then upload data documents to be processed with that styling applied.

New User Workflow

1. User uploads a style template (DOCX or PDF)
↓
2. System extracts and displays styling information
↓
3. User activates/confirms the template
↓
4. User uploads data documents to process
↓
5. System processes with template styling applied
↓
6. User downloads beautifully formatted reports

API Endpoints Reference

Template Management Endpoints

1. Upload Template

Endpoint: `POST /templates/upload`

```
Headers: Content-Type: multipart/form-data
Body:
- file: [DOCX or PDF file]
- template_name: [optional string, defaults to filename]

Response:
{
  "success": true,
  "template_name": "professional",
  "template_path": "/path/to/template",
  "message": "Template 'professional' registered successfully",
  "pdf_converted": false,
  "conversion_method": null,
  "styles": {
    "title_font": {
      "name": "Calibri",
      "size": 28,
      "bold": true
    },
    "heading_font": {
      "name": "Calibri",
      "size": 16,
      "bold": true
    },
    "body_font": {
      "name": "Calibri",
      "size": 12
    }
  }
}
```

Note: If a PDF is uploaded, `pdf_converted` will be `true` and `conversion_method` will indicate the conversion method used ("pdf2docx" or "fallback").

2. List All Templates

Endpoint: `GET /templates`

```
Response:
{
  "templates": ["professional", "minimal", "corporate"],
  "active_template": "professional",
  "total_templates": 3
}
```

3. Activate Template

Endpoint: `POST /templates/{template_name}/activate`

```
URL: /templates/professional/activate
Method: POST

Response:
{
  "success": true,
  "active_template": "professional",
  "message": "Template 'professional' activated",
  "profile": {
    "template_name": "professional",
    "title_font": "Calibri",
    "heading_font": "Calibri",
    "body_font": "Calibri"
  }
}
```

4. Get Template Info

Endpoint: GET /templates/info/{template_name}

URL: /templates/info/professional

Response:

```
{
  "template_name": "professional",
  "template_path": "/path/to/template.docx",
  "margins": {
    "left": 1440,
    "right": 1440,
    "top": 1440,
    "bottom": 1440
  },
  "fonts": {
    "title": {
      "name": "Calibri",
      "size": 28,
      "bold": true,
      "italic": false,
      "underline": false,
      "color": "003366",
      "highlight_color": null,
      "strike_through": false,
      "subscript": false,
      "superscript": false
    },
    "heading": {
      "name": "Calibri",
      "size": 16,
      "bold": true,
      "italic": false,
      "underline": false,
      "color": "003366",
      "highlight_color": null,
      "strike_through": false,
      "subscript": false,
      "superscript": false
    },
    "body": {
      "name": "Calibri",
      "size": 12,
      "bold": false,
      "italic": false,
      "underline": false,
      "color": "000000",
      "highlight_color": null,
      "strike_through": false,
      "subscript": false,
      "superscript": false
    }
  },
  "paragraph_style": {
    "alignment": "left",
    "line_spacing": 1.15,
    "space_before": 0,
    "space_after": 0,
    "indent_left": 0,
    "indent_right": 0,
    "indent_first_line": 0,
    "keep_with_next": false,
    "page_break_before": false,
    "widow_control": true,
    "shading_color": null
  },
  "table_style": {
    "style_name": "Light Grid Accent 1",
    "border_color": "000000",
    "border_width": 1,
    "cell_padding": 0,
    "header_background_color": "4472C4",
    "header_font": {
      "name": "Calibri",
      "size": 11,
      "bold": true,
      "italic": false,
      "underline": false,
      "color": "FFFFFF",
      "highlight_color": null,
      "strike_through": false,
      "subscript": false,
      "superscript": false
    }
  }
}
```

```
    },
    "body_font": {
      "name": "Calibri",
      "size": 11,
      "bold": false,
      "italic": false,
      "underline": false,
      "color": "000000",
      "highlight_color": null,
      "strike_through": false,
      "subscript": false,
      "superscript": false
    },
    "banding": true,
    "banding_color": "D9E1F2"
  },
  "list_styles": {},
  "image_styles": [],
  "logo_style": {
    "width": 0,
    "height": 0,
    "alignment": "left",
    "wrap_text": true,
    "position_x": 0,
    "position_y": 0
  },
  "logo_path": null,
  "page": {
    "width": 8.5,
    "height": 11.0,
    "orientation": "portrait"
  },
  "header_content": "",
  "footer_content": "",
  "has_page_numbers": false,
  "color_scheme": {
    "primary": "003366",
    "secondary": "003366",
    "body": "000000"
  },
  "implied_rules": [],
  "template_insights": {
    "headings": [],
    "table_headers": [],
    "has_logo": false,
    "total_paragraphs": 0,
    "total_tables": 0,
    "total_images": 0
  },
  "custom_styles_count": 0
}
```

Frontend Implementation Steps

Step 1: Create Template Management UI Component

Create a new React/Vue component for template management:

File: `components/TemplateManager.jsx` (or `.vue`)

```

import React, { useState, useEffect } from 'react';
import { uploadFile, getTemplateList, activateTemplate, getTemplateInfo } from '../api/vercelApiClient';

export function TemplateManager({ onTemplateActivated }) {
  const [templates, setTemplates] = useState([]);
  const [activeTemplate, setActiveTemplate] = useState(null);
  const [selectedTemplate, setSelectedTemplate] = useState(null);
  const [uploadingTemplate, setUploadingTemplate] = useState(false);
  const [templateInfo, setTemplateInfo] = useState(null);
  const [error, setError] = useState(null);

  // Load existing templates on mount
  useEffect(() => {
    loadTemplates();
  }, []);

  const loadTemplates = async () => {
    try {
      const result = await getTemplateList();
      setTemplates(result.templates || []);
      setActiveTemplate(result.active_template);
    } catch (err) {
      setError(`Failed to load templates: ${err.message}`);
    }
  };

  const handleTemplateUpload = async (file) => {
    if (!file) return;

    setUploadingTemplate(true);
    setError(null);

    try {
      const templateName = file.name.replace('.docx', '').replace('.pdf', '');
      const response = await uploadTemplate(file, templateName);

      // Refresh template list
      await loadTemplates();

      setSelectedTemplate(response.template_name);
      alert(`✅ Template "${response.template_name}" uploaded successfully!`);
    } catch (err) {
      setError(`Failed to upload template: ${err.message}`);
    } finally {
      setUploadingTemplate(false);
    }
  };

  const handleTemplateActivate = async (templateName) => {
    try {
      const response = await activateTemplate(templateName);
      setActiveTemplate(response.active_template);

      // Get template details
      const info = await getTemplateInfo(templateName);
      setTemplateInfo(info);

      if (onTemplateActivated) {
        onTemplateActivated(response);
      }

      alert(`✅ Template "${templateName}" activated!`);
    } catch (err) {
      setError(`Failed to activate template: ${err.message}`);
    }
  };

  return (
    <div className="template-manager">
      <h2> 📄 Style Template Management</h2>

      {/* Upload Section */}
      <div className="template-upload-section">
        <h3>Upload New Template</h3>
        <p>Upload a Word document (.docx) or PDF that defines your report styling</p>

        <input
          type="file"
          accept=".docx,.pdf"

```

```

        onChange={(e) => handleTemplateUpload(e.target.files[0])}
        disabled={uploadingTemplate}
      />

      {uploadingTemplate && <p>🔄 Uploading template...</p>}
      {error && <p className="error">❌ {error}</p>}}
    </div>

    /* Template List Section */
    <div className="template-list-section">
      <h3>Available Templates</h3>

      {templates.length === 0 ? (
        <p>No templates uploaded yet. Upload a template above to get started.</p>
      ) : (
        <ul className="template-list">
          {templates.map((template) => (
            <li
              key={template}
              className={`template-item ${template === activeTemplate ? 'active' : ''}`}
            >
              <span className="template-name">{template}</span>

              {template === activeTemplate ? (
                <span className="badge active-badge">✓ Active</span>
              ) : (
                <button
                  onClick={() => handleTemplateActivate(template)}
                  className="activate-btn"
                >
                  Activate
                </button>
              )}
            </li>
          )]}
        </ul>
      )}

    </div>

    /* Active Template Info Section */
    {templateInfo && (
      <div className="template-info-section">
        <h3>📄 Active Template Details</h3>

        /* PDF Conversion Status */
        {templateInfo.pdf_converted && (
          <div className="conversion-notice">
            <span className="badge info-badge">PDF Converted</span>
            <span>Conversion method: {templateInfo.conversion_method}</span>
          </div>
        )}

        <div className="info-grid">
          /* Title Font */
          <div className="info-card">
            <h4>Title Font</h4>
            <p><strong>Font:</strong> {templateInfo.fonts.title.name}</p>
            <p><strong>Size:</strong> {templateInfo.fonts.title.size}pt</p>
            <p><strong>Bold:</strong> {templateInfo.fonts.title.bold ? 'Yes' : 'No'}</p>
            <p><strong>Italic:</strong> {templateInfo.fonts.title.italic ? 'Yes' : 'No'}</p>
            <p><strong>Underline:</strong> {templateInfo.fonts.title.underline ? 'Yes' : 'No'}</p>
            <p><strong>Color:</strong> #{templateInfo.fonts.title.color}</p>
            {templateInfo.fonts.title.highlight_color && (
              <p><strong>Highlight:</strong> #{templateInfo.fonts.title.highlight_color}</p>
            )}
          </div>

          /* Heading Font */
          <div className="info-card">
            <h4>Heading Font</h4>
            <p><strong>Font:</strong> {templateInfo.fonts.heading.name}</p>
            <p><strong>Size:</strong> {templateInfo.fonts.heading.size}pt</p>
            <p><strong>Bold:</strong> {templateInfo.fonts.heading.bold ? 'Yes' : 'No'}</p>
            <p><strong>Italic:</strong> {templateInfo.fonts.heading.italic ? 'Yes' : 'No'}</p>
            <p><strong>Color:</strong> #{templateInfo.fonts.heading.color}</p>
          </div>

          /* Body Font */
          <div className="info-card">
            <h4>Body Font</h4>

```

```

    <p><strong>Font:</strong> {templateInfo.fonts.body.name}</p>
    <p><strong>Size:</strong> {templateInfo.fonts.body.size}pt</p>
    <p><strong>Bold:</strong> {templateInfo.fonts.body.bold ? 'Yes' : 'No'}</p>
    <p><strong>Color:</strong> #{templateInfo.fonts.body.color}</p>
  </div>

  {/ Paragraph Style */}
  <div className="info-card">
    <h4>Paragraph Style</h4>
    <p><strong>Alignment:</strong> {templateInfo.paragraph_style.alignment}</p>
    <p><strong>Line Spacing:</strong> {templateInfo.paragraph_style.line_spacing}</p>
    <p><strong>Space Before:</strong> {templateInfo.paragraph_style.space_before}</p>
    <p><strong>Space After:</strong> {templateInfo.paragraph_style.space_after}</p>
    <p><strong>Indent First:</strong> {templateInfo.paragraph_style.indent_first_line}</p>
  </div>

  {/ Table Style */}
  <div className="info-card">
    <h4>Table Style</h4>
    <p><strong>Style:</strong> {templateInfo.table_style.style_name}</p>
    <p><strong>Border Color:</strong> #{templateInfo.table_style.border_color}</p>
    <p><strong>Header BG:</strong> #{templateInfo.table_style.header_background_color || 'None'}</p>
    <p><strong>Banding:</strong> {templateInfo.table_style.banding ? 'Yes' : 'No'}</p>
    {templateInfo.table_style.banding_color && (
      <p><strong>Banding Color:</strong> #{templateInfo.table_style.banding_color}</p>
    )}
  </div>

  {/ Page Settings */}
  <div className="info-card">
    <h4>Page Settings</h4>
    <p><strong>Width:</strong> {templateInfo.page.width}"</p>
    <p><strong>Height:</strong> {templateInfo.page.height}"</p>
    <p><strong>Orientation:</strong> {templateInfo.page.orientation}</p>
  </div>

  {/ Template Insights */}
  <div className="info-card">
    <h4>Template Insights</h4>
    <p><strong>Paragraphs:</strong> {templateInfo.template_insights.total_paragraphs}</p>
    <p><strong>Tables:</strong> {templateInfo.template_insights.total_tables}</p>
    <p><strong>Images:</strong> {templateInfo.template_insights.total_images}</p>
    <p><strong>Has Logo:</strong> {templateInfo.template_insights.has_logo ? 'Yes' : 'No'}</p>
    <p><strong>Custom Styles:</strong> {templateInfo.custom_styles_count}</p>
  </div>

  {/ Color Scheme */}
  <div className="info-card">
    <h4>Color Scheme</h4>
    <p><strong>Primary:</strong> #{templateInfo.color_scheme.primary}</p>
    <p><strong>Secondary:</strong> #{templateInfo.color_scheme.secondary}</p>
    <p><strong>Body:</strong> #{templateInfo.color_scheme.body}</p>
  </div>
</div>
</div>
  )}
</div>
  );
}

```

Step 2: Update API Client Helper

Update `vercelApiClient.js` to include template endpoints and generation mode:


```
// Add these functions to vercelApiClient.js

export async function uploadTemplate(file, templateName) {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('template_name', templateName);

  const response = await fetch(`${getApiBaseUrl()}/templates/upload`, {
    method: 'POST',
    body: formData,
  });

  if (!response.ok) {
    throw new Error(`Upload failed: ${response.statusText}`);
  }

  return response.json();
}

export async function getTemplateList() {
  const response = await fetch(`${getApiBaseUrl()}/templates`);

  if (!response.ok) {
    throw new Error(`Failed to fetch templates: ${response.statusText}`);
  }

  return response.json();
}

export async function activateTemplate(templateName) {
  const response = await fetch(
    `${getApiBaseUrl()}/templates/${templateName}/activate`,
    { method: 'POST' }
  );

  if (!response.ok) {
    throw new Error(`Failed to activate template: ${response.statusText}`);
  }

  return response.json();
}

export async function getTemplateInfo(templateName) {
  const response = await fetch(
    `${getApiBaseUrl()}/templates/info/${templateName}`
  );

  if (!response.ok) {
    throw new Error(`Failed to fetch template info: ${response.statusText}`);
  }

  return response.json();
}

export async function startProcessing(generationMode = 'apply') {
  const formData = new FormData();
  formData.append('generation_mode', generationMode);

  const response = await fetch(`${getApiBaseUrl()}/process`, {
    method: 'POST',
    body: formData,
  });

  if (!response.ok) {
    throw new Error(`Failed to start processing: ${response.statusText}`);
  }

  return response.json();
}

```

Step 3: Integrate into Main Application

Update your main application component to include the template manager and generation mode selection:

```

import { TemplateManager } from './components/TemplateManager';
import { DocumentUploader } from './components/DocumentUploader';
import { ProcessingStatus } from './components/ProcessingStatus';

export function App() {
  const [templateActivated, setTemplateActivated] = useState(false);
  const [processing, setProcessing] = useState(false);
  const [generationMode, setGenerationMode] = useState('apply'); // 'apply' or 'modify'

  return (
    <div className="app-container">
      <h1>🇳🇬 NigeriaCompliance - Template-Based Report Generator</h1>

      {/* Step 1: Template Management */}
      <section className="template-section">
        <TemplateManager
          onTemplateActivated={() => setTemplateActivated(true)}
        />
      </section>

      {/* Step 2: Generation Mode Selection (only show if template is active) */}
      {templateActivated && (
        <section className="generation-mode-section">
          <h3>Document Generation Mode</h3>
          <div className="mode-selector">
            <label className="mode-option">
              <input
                type="radio"
                value="apply"
                checked={generationMode === 'apply'}
                onChange={(e) => setGenerationMode(e.target.value)}
              />
              <div className="mode-details">
                <strong>Apply Styles to New Document</strong>
                <p>Extract styles from template and apply to new document</p>
              </div>
            </label>
            <label className="mode-option">
              <input
                type="radio"
                value="modify"
                checked={generationMode === 'modify'}
                onChange={(e) => setGenerationMode(e.target.value)}
              />
              <div className="mode-details">
                <strong>Modify Template In-Place (Recommended)</strong>
                <p>Modify the template document directly for most accurate mirroring</p>
              </div>
            </label>
          </div>
        </section>
      )}

      {/* Step 3: Document Upload (only show if template is active) */}
      {templateActivated && (
        <section className="upload-section">
          <DocumentUploader
            onProcessingStart={() => setProcessing(true)}
            onProcessingComplete={() => setProcessing(false)}
            generationMode={generationMode}
          />
        </section>
      )}

      {/* Step 4: Processing Status */}
      {processing && (
        <section className="status-section">
          <ProcessingStatus />
        </section>
      )}
    </div>
  );
}

```

Step 4: CSS Styling

Add styling to `styles/templateManager.css` :

```
.template-manager {
  background: #f9f9f9;
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 20px;
  border: 1px solid #e0e0e0;
}

.template-upload-section {
  margin-bottom: 20px;
}

.template-upload-section input {
  display: block;
  padding: 10px;
  border: 2px dashed #007bff;
  border-radius: 4px;
  cursor: pointer;
  margin: 10px 0;
}

.template-list {
  list-style: none;
  padding: 0;
}

.template-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px;
  margin: 5px 0;
  background: white;
  border: 1px solid #ddd;
  border-radius: 4px;
  border-left: 4px solid #007bff;
}

.template-item.active {
  border-left-color: #28a745;
  background: #f0fff4;
}

.template-name {
  font-weight: 500;
}

.badge {
  padding: 4px 8px;
  border-radius: 12px;
  font-size: 0.85em;
}

.active-badge {
  background: #28a745;
  color: white;
}

.activate-btn {
  padding: 6px 12px;
  background: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.9em;
}

.activate-btn:hover {
  background: #0056b3;
}

.template-info-section {
  margin-top: 20px;
  padding: 15px;
  background: white;
  border-radius: 4px;
}
```

```
.info-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 15px;
  margin-top: 15px;
}

.info-card {
  padding: 12px;
  border: 1px solid #e0e0e0;
  border-radius: 4px;
  background: #fafafa;
}

.info-card h4 {
  margin: 0 0 10px 0;
  color: #333;
  border-bottom: 2px solid #007bff;
  padding-bottom: 5px;
}

.info-card p {
  margin: 5px 0;
  font-size: 0.9em;
}

.error {
  color: #dc3545;
  padding: 10px;
  background: #f8d7da;
  border: 1px solid #f5c6cb;
  border-radius: 4px;
  margin: 10px 0;
}

.generation-mode-section {
  margin-bottom: 20px;
  padding: 15px;
  background: white;
  border-radius: 4px;
  border: 1px solid #e0e0e0;
}

.mode-selector {
  display: flex;
  gap: 20px;
  margin-top: 15px;
}

.mode-option {
  flex: 1;
  display: flex;
  align-items: flex-start;
  gap: 10px;
  padding: 15px;
  border: 2px solid #e0e0e0;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.2s;
}

.mode-option:hover {
  border-color: #007bff;
  background: #f8f9fa;
}

.mode-option input[type="radio"] {
  margin-top: 5px;
}

.mode-details strong {
  display: block;
  margin-bottom: 5px;
  color: #333;
}

.mode-details p {
  margin: 0;
  font-size: 0.9em;
  color: #666;
}
```

```
    line-height: 1.4;
  }

  .conversion-notice {
    display: flex;
    align-items: center;
    gap: 10px;
    margin-bottom: 15px;
    padding: 10px;
    background: #e7f3ff;
    border-radius: 4px;
    border-left: 4px solid #007bff;
  }

  .info-badge {
    background: #007bff;
    color: white;
    padding: 4px 8px;
    border-radius: 12px;
    font-size: 0.85em;
  }
}
```

User Instructions for Frontend

For End Users

1. Upload a Template Document

- Click "  Upload Template"
- Select a Word document (.docx) or PDF that has your desired styling
- The system will automatically extract fonts, colors, and formatting

2. Confirm Template Details

- Review the extracted styling information
- Click "Activate" to use this template

3. Upload Data Documents

- Once template is active, upload your source documents
- Select the department (Finance, HR, Procurement, Operations)
- Click "Upload"

4. Process Documents

- Click "Start Processing"
- Wait for processing to complete
- Download the beautifully formatted reports

Technical Notes

- Templates support both DOCX and PDF formats
 - PDF templates are automatically converted to DOCX for enhanced extraction
 - Extracted styling includes: fonts (underline, highlight, strike-through, subscript, superscript), paragraph styles (indentation, spacing, page breaks, shading), table styles (borders, shading, header/body fonts, banding), list styles, image styles
 - Multiple templates can be registered but only one can be active
 - Template profiles are cached locally for performance
 - Templates can be reused across multiple processing runs
 - Two generation modes available:
 - **Apply:** Extract styles and apply to new document
 - **Modify:** Modify template in-place for most accurate mirroring (recommended)
 - Template insights provide metrics on template structure (paragraphs, tables, images, custom styles)
-

Error Handling

Add proper error handling for these scenarios:

```
const errorCases = {  
  TEMPLATE_NOT_FOUND: "Template not found. Please upload a template first.",  
  INVALID_FILE_FORMAT: "Invalid file format. Please upload a .docx or .pdf file.",  
  UPLOAD_FAILED: "Template upload failed. Please check the file and try again.",  
  ACTIVATION_FAILED: "Failed to activate template. Please try again.",  
  NO_TEMPLATES: "No templates available. Please upload a template first.",  
};
```

Testing Checklist

- ☐ [] Template upload works for DOCX files
 - ☐ [] Template upload works for PDF files
 - ☐ [] PDF conversion status is displayed correctly
 - ☐ [] Template list displays all uploaded templates
 - ☐ [] Template activation updates the active template
 - ☐ [] Template info displays enhanced styling details (fonts, paragraph, table, list, image styles)
 - ☐ [] Template insights display correctly (paragraphs, tables, images, custom styles)
 - ☐ [] Generation mode selection works (apply/modify)
 - ☐ [] Generation mode is passed to processing endpoint
 - ☐ [] Error handling for invalid files
 - ☐ [] Error handling for network failures
 - ☐ [] Processing starts and completes with template active
 - ☐ [] Generated reports use template styling
 - ☐ [] Multiple templates can be managed independently
 - ☐ [] Color scheme displays correctly
 - ☐ [] Table style details display correctly (borders, shading, banding)
-

Deployment Notes

1. Ensure the API endpoints are accessible from the frontend URL
2. Update `vercelApiClient.js` with correct API base URL
3. Configure CORS if frontend and backend are on different domains
4. Test with both development and production API endpoints